

PathFinder - Comprehensive Developer Guide 🚀

PathFinder is a highly modular, full-stack platform designed to bridge the gap between **Students** looking for opportunities and **Organizations** offering them (Internships, Scholarships, Grants, Hackathons).

This guide serves as a deep dive into the architecture, features, and individual modules of the application for developers looking to understand, maintain, or expand the codebase.

🔧 Technology Stack

Frontend Architecture

- **React.js (Vite):** Lightning-fast development environment and optimized production builds.
- **TypeScript:** Strict static typing across all components and API responses.
- **Custom Design System:** Built entirely with Vanilla CSS using CSS Variables (`index.css`), ensuring a premium, glassmorphic aesthetic without relying on external UI libraries like Tailwind or Bootstrap.
- **React Router DOM:** Client-side routing for seamless page transitions.
- **Context API:** Global state management for Authentication (`AuthContext`) and Theming (`ThemeContext`).

Backend Architecture

- **Node.js & Express.js:** RESTful API architecture divided into modular route handlers.
 - **TypeScript:** End-to-end type safety.
 - **Prisma ORM:** Type-safe database client.
 - **SQLite:** Lightweight database (easily swappable for PostgreSQL in production via Prisma).
 - **JWT (JSON Web Tokens):** Secure, stateless authentication system.
 - **Multer:** Middleware for handling `multipart/form-data` (file uploads for resumes and avatars).
-

🔗 Module-by-Module Breakdown

The codebase is highly modular, with clear separation of concerns between the frontend pages/components and backend route handlers.

1. Authentication Module (`/api/auth`)

Handles all user registration, login, and session management.

- **Roles:** Users register as either `STUDENT` or `ORGANIZATION`. Admins (`ADMIN`) are created manually in the database.
- **JWT Flow:** Upon login, a JWT is signed with the user's ID and Role. This token must be passed in the `Authorization: Bearer <token>` header for all protected routes.
- **Password Reset:** Generates secure tokens saved to the database. Emails are sent out containing a link to reset the password.

2. Smart Matching Engine (`/api/applications/match`)

One of the core features of PathFinder is the intelligent matching algorithm that connects students to the right opportunities.

- **How it works:** When a student applies for or views an opportunity, the backend compares the student's listed skills, major, and graduation year against the opportunity's requirements.
- **Scoring:** It calculates a `matchScore` (0-100%). It checks for exact skill matches, partial category matches, and academic alignment.
- **Insights:** The system generates a `matchReason` string (e.g., "Strong match based on React and Node.js skills") to give both the student and organization context on *why* they are a good fit.

3. Student Module (`/api/students`)

Handles the complete lifecycle of a student user.

- **Profiles:** Students can upload their resumes (PDFs) and profile pictures. Data includes universities, majors, graduation years, and a dynamic array of skills.
- **Bookmarking:** Students can bookmark opportunities for later. The frontend uses a toggle state that syncs with the `Bookmark` database table.
- **Dashboards:** The student dashboard aggregates data: how many applications they've submitted, their average match score, and their recent activity.

4. Organization Module (`/api/org`)

Handles the lifecycle of organizations (companies, universities, non-profits).

- **Profiles:** Organizations can set up a public profile with a logo, description, and industry tags.
- **Applicant Tracking:** Organizations have a dedicated dashboard to review applications. They can see the `matchScore` of every applicant, download their resume directly, and update the application status (`PENDING`, `REVIEWING`, `ACCEPTED`, `REJECTED`).

5. Opportunities Module (/api/opportunities)

The core data entity of the platform.

- **Categories:** Supports `INTERNSHIP`, `SCHOLARSHIP`, `GRANT`, and `HACKATHON`.
- **Filtering & Search:** The frontend `/search` page allows complex filtering by category, location (`Remote/On-site`), and keyword matching against the opportunity title and description.
- **Status:** Organizations can save opportunities as `DRAFT` or publish them as `PUBLISHED`.

6. Support Ticketing System (/api/tickets)

A built-in customer service module.

- **Creation:** Any user can create a ticket categorized by `BUG_REPORT`, `FEATURE_REQUEST`, etc.
- **Threaded Messages:** Users and Admins can converse back-and-forth within a specific ticket.
- **Admin Dashboard:** Admins have a global view of all tickets, can assign priorities, and update statuses to `RESOLVED` to close the loop.

7. AI Assistant Module (/api/ai)

An intelligent chatbot integrated into the frontend to guide students.

- **Integration:** Connects securely to an external LLM API from the backend (API keys are kept strictly in the `.env` file and never exposed to the client).
- **Context-Aware:** The backend injects the student's profile data (skills, major) into the system prompt so the AI can give personalized career advice and recommend specific types of opportunities.

8. Notification System (/api/notifications)

Keeps users engaged.

- **Triggers:** Automatically generates notifications when application statuses change, when organizations receive new applicants, or when tickets receive replies.
- **Unread Tracking:** Tracks `isRead` boolean flags to display badges in the frontend sidebar.

Database Schema (Prisma)

The database relies on strong relational integrity. Key models include:

- **User:** The base table containing authentication data and roles.
- **StudentProfile / OrganizationProfile:** 1-to-1 relations to the `User` table containing role-specific data.
- **Opportunity:** Created by Organizations (1-to-many).
- **Application:** The join table between `User` (Student) and `Opportunity`. Contains the calculated `matchScore`.
- **Ticket / TicketMessage:** 1-to-many relation for the support system.
- **Notification:** Tied to a specific `User`.

Security Best Practices Implemented

1. **Frontend Route Protection:** `App.tsx` wraps sensitive routes in a `<ProtectedRoute>` component. If a student tries to access an organization URL (e.g., `/org/dashboard`), they are immediately redirected.
2. **Backend Authorization Middleware:** Every API endpoint uses an `authenticateToken` middleware. Admin-only endpoints use an additional `requireRole('ADMIN')` check. A frontend bypass will still result in a `403 Forbidden` from the API.
3. **Password Hashing:** All passwords are encrypted using `bcrypt` before being stored in the database.
4. **Environment Variables:** All secrets (JWT Keys, AI API Keys, Database URLs) are strictly loaded via `dotenv` in the backend and never committed to source control.

Setup Instructions

1. Backend Setup

```
cd backend
npm install
npx prisma generate
npx prisma db push
npm run dev
```

Backend .env file required:

```
PORT=5000
DATABASE_URL="file:./dev.db"
JWT_SECRET="<generate-a-secure-random-string>"
AI_API_KEY="<your-ai-provider-api-key>"
EMAIL_USER="<smtp-email-address>"
EMAIL_PASS="<smtp-password>"
```

2. Frontend Setup

```
cd frontend
npm install
npm run dev
```

Frontend .env file required:

```
VITE_API_URL="http://localhost:5000"
```

End of Developer Guide.